PROTOTYP AV E-HANDELSAPPLIKATION

Introduktion

Bokhyllan är en prototyp av en digital bokhandel. Tema är valt utifrån min kärlek för litteratur och min glädje när jag vistas i en vacker (fysisk) bokhandel. Visionen är att erbjuda en välkomnande och tilltalande digital bokhandel. Samtidigt ska det vara enkelt för besökaren att leta efter och hitta böcker, att skapa och hantera sitt konto och genomföra köp. **Bokhyllan** är byggd som en ”single page application, SPA” med de senaste ramverken och verktygen inom applikationsutveckling tillsammans med en robust och säker hantering av data i en välkänd databas.

Design

Projektets började inuti mitt huvud och ganska snart konkretiserades mina idéer med hjälp av omvärldsanalys och Figma (bild 1 a och b). Omvärldsanalysen avgränsade jag till två exempel – det ena tilltalas jag av stilmässigt men kanske inte innehållsmässigt och det andra tilltalas jag inte av stilmässigt men definitivt innehållsmässigt. Till omvärldsspaningen lade jag en sökning efter färdiga förslag på färgpaletter (coolers.co) som också bedöms utifrån tillgänglighet. Jag valde mellan två närliggande, och lät tillgänglighetsbetyget avgöra. Jag tog också med den mörkaste färgen från den bortvalda paletten. I Figma fortsatte konkretiseringen genom de wireframes (bild 2) tillsammans med alla olika komponenter som växte fram och slutligen en enkel skiss över ett tänkt användarflöde (bild 3).

Arkitektur

Applikationen har en modulär arkitektur i flera lager för att leva upp till principen om att separera ansvar och uppgifter (*separation of concerns*). Principen tillämpas i form av modeller, kontrollanter och rutter i backend (bild 4), och i form av sidor, sektioner och komponenter i frontend. Dessutom används mellanvara för att hantera autentisering (tokenHandlers i backend) samt datakontexter (autentiseringswrapper, contexts, contextProviders) med ”krokar” (hooks) i frontend. Genom att följa den första principen blir det enklare att felsöka och underhålla koden, men också att skala om den när applikationen behöver krympa eller växa. Genom denna moduläritet följer applikationen också principen om omvänt beroende (*dependency inversion*). Detta gör applikationen mindre sårbar för otillbörligt intrång. Ett API, som definieras i frontend, knyter samman frontend och backend, där autentisering också sker för en del av innehållet och för vissa funktioner. Applikationens uppbyggnad syns också i projektets mappstruktur (bild 5) med en projektdel för frontend och en projektdel för backend.

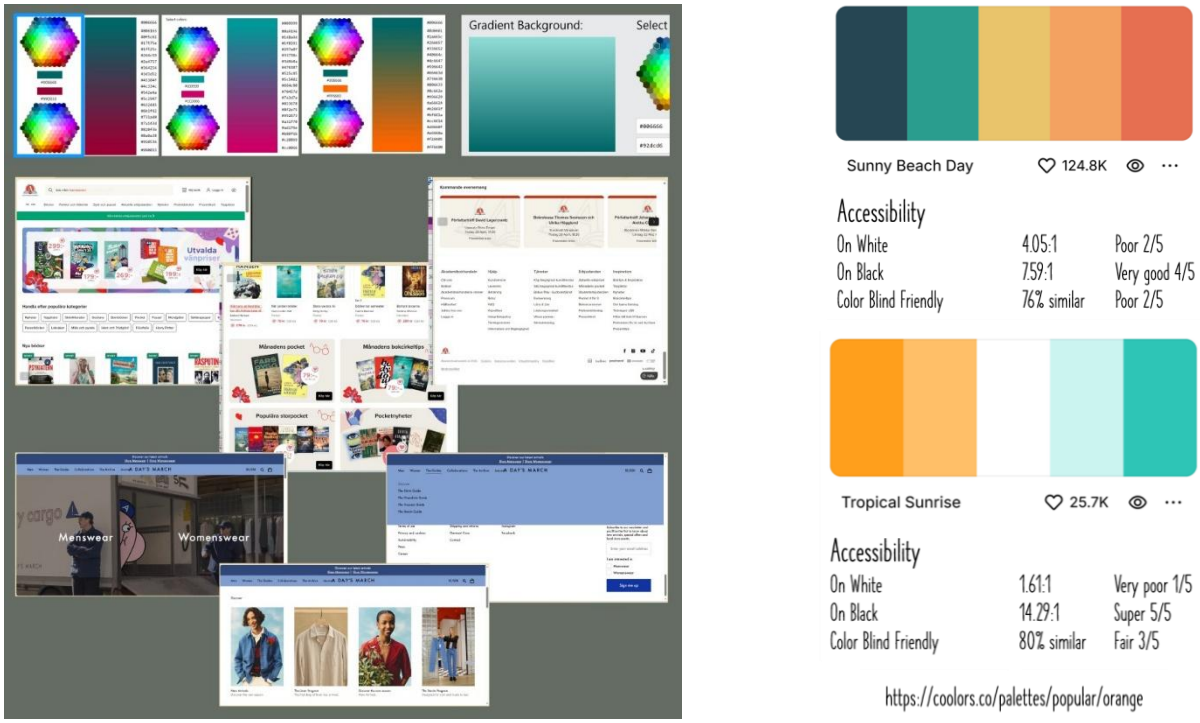


Bild 1. Skärmdump av stilguide från Figma (vänster) och färgpaletter från coolers.co (höger).

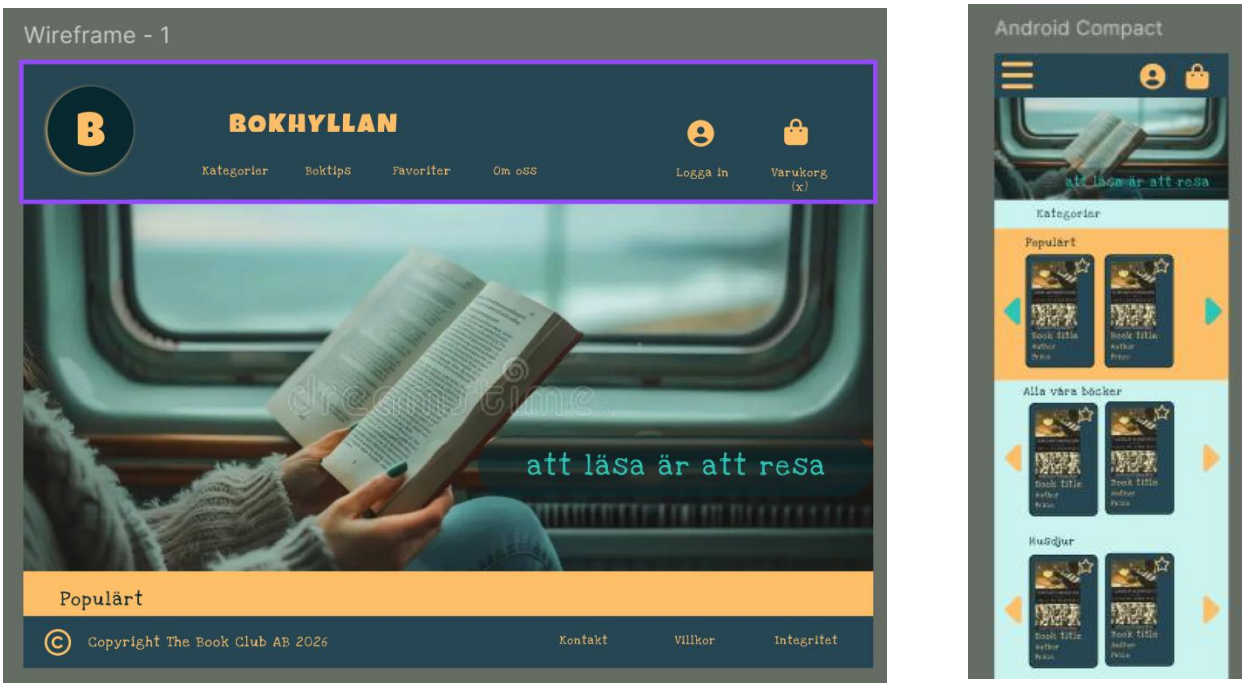


Bild 2. Wireframes för desktop (vänster) och mobil (höger) från Figma.

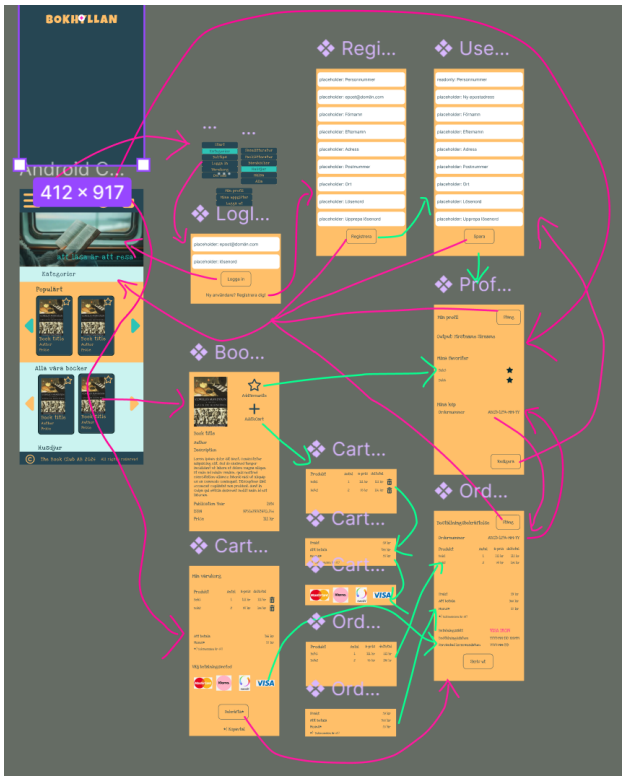


Bild 3. Användarflöden illustrerade i Figma.

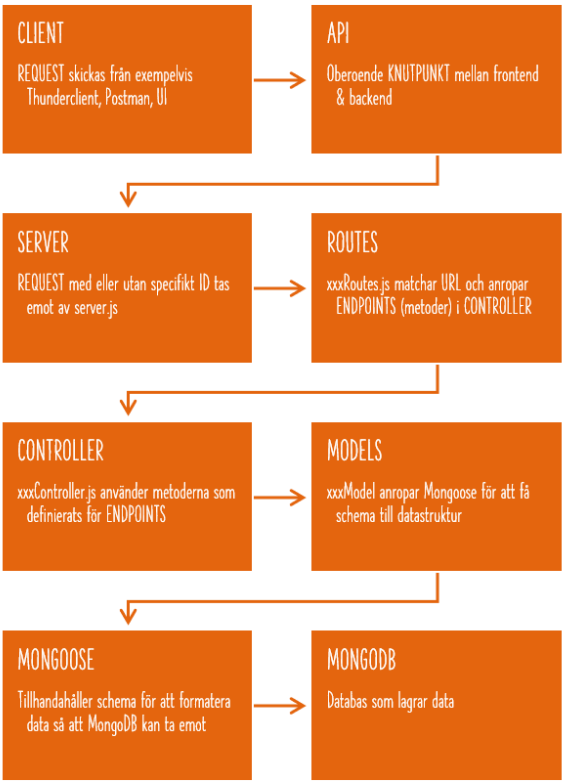


Bild 4. Dataflöden i applikationen.

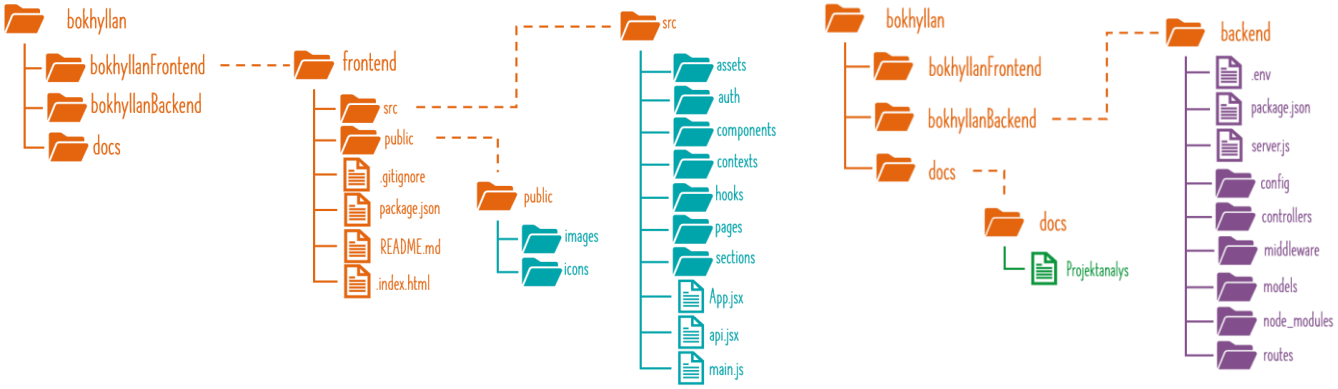


Bild 5. Den övergripande mapp- och filstrukturen för frontend (vänster) och backend (höger).

Teknikstack

Applikationens frontend är byggd med React och Vite

Ramverket React är ett bibliotek och en arkitektur för webb- och native-användargränssnitt. React installerades med JavaScript och React-kompilator som tillval genom Vite. Vite är ett frontend-byggverktyg med stöd för rendering på serversidan (SSR) och som möjliggör omedelbar ”hot module replacement” (HMR) så att utvecklaren slipper ladda om manuellt.

Applikationens backend är byggd med Node.js

Node.js är en plattformsoberoende körmiljö för JavaScript oh kan användas för att bygga servrar, webbappar, med mera. Nodemon är ett verktyg för Node.js som håller koll på projektfilerna och startar om applikationen automatiskt när ändringar görs i filerna under utvecklingsfasen. Dotenv är en oberoende Node.js-modul som läser in (eller laddar) de miljövariabler som definierats i en .env-fil och skickar till process.env. Detta håller API-nycklar och databasuppgifter utanför källkoden och ger en säkrare applikation. Express är ett ramverk för webb- och mobilappar som underlättar API-bygge och fungerar som en mellanvara som anropas i appservern.

MongoDB används som databas

Backenden arbetar mot ett databaskluster för användar-, produkt- och orderdata som tillhandahålls av MongoDB Atlas. Mongoose är en Object Document Mapper (ODM) för MongoDB som översätter data från appmodellers format till ett format som krävs av MongoDB.

Tillägg och verktyg

- React Router är ett standardbibliotek för routing i React-webbapplikationer, som möjliggör navigering mellan komponenter utan att sidan laddas om.
- Express-async-handler är ett paket i form av mellanvara som kan användas för att förenkla felhantering i asynkrona Express-rutthanterare.
- Cross Origin Resource Sharing (CORS) är en webbläsarsäkerhetsmekanism som genom HTTP-headers kan användas för att specificera åtkomst till resurserna på appens API-server utöver de som definieras av standardpolicyn.
- BCrypt tillhandahåller hashade lösenord tillsammans med SALT som itererar hashningen för att ge högre säkerhet.
- JSON Web Token (JWT) tillhandahåller unika användarsignaturer i JSON-format som en del i säkerheten.

Tester

Under projektet har en beteendedriven utvecklingsapproach tillämpats med kontinuerliga tester av användarens input i UI:t. API-tester har gjorts med Thunder Client. Förutsättningar och testidentitet för testkörning beskrivs i README.md i GitHub-repot.

Förbättringsförslag

Applikationen i sitt befintliga skick uppfyller kraven för inlämningsuppgiften både avseende godkänt och väl godkänt. Men självklart är det mycket som kvarstår att utveckla för att uppnå en nivå som motsvara en publicerbar applikation. Många sidor har endast en rubrik och vissa funktioner är inte aktiverade. Därtill utnyttjas inte exempelvis komponenterna i mellanvaran för hantering av favoriter till fullo. Vad gäller

användarvänlighet och tillgänglighet har jag tagit mig friheten att trots de hänsyn jag tagit (se design, ovan) ha en suboptimal färgsättning och ”roliga” typsnitt. I en app i produktion skulle jag använda mig av färgschemat så att mer kontrast skapas mellan bakgrundsfärgen och mindre objekt, som text och ikoner. Dessutom skulle jag använda tydligare typsnitt.

Andra förbättringsåtgärder är att lägga till ett sökfält där användaren kan skriva in några bokstäver eller ord i en boktitel. Kanske också ändra filtreringen till en mera sammanhållen variant för alla tre ”kategorilagren”. BookCards, som visar lite grundinformation om en bok bör struktureras bättre, så att utseendet harmoniseras mellan korten i karusellerna oberoende av längden på titel och namn. Stilen på komponenten för favoriter behöver uppdateras då den hamnar knasigt nu i Profilvyn. Nu har jag dubbel uppsättning av betalnings- och leveransalternativ på grund av autentiserade och inte autentiserad användare – Detta kan göras snyggare.

Jag vill utveckla navigationen genom att byta från <Link> till <NavLink> så att användaren kan se vilken flik som är aktiv. Jag kanske också skulle utveckla mer automatik i navigationen exempelvis genom att omdirigera användaren direkt till start efter orderbekräftelse eller skapa ett popupfönster som användaren kan stänga. Jag vill skapa en global errorHandler, utvekla darkmode-stil och utveckla den funktionalitet som saknas i nuläget, exempelvis i köpflödet ska användarinformation importeras för inloggad användare och beställningsinformation importeras i bekräftelsen, som är det sista steget i köpflödet.

Utmaningar

Den största utmaningen under projektet har varit de många mapparna och filerna. Jag upplever det fortfarande som väldigt komplext och svårt att överblicka och jag går lätt vilse. Trots denna utmaning har jag njutit av varje steg i processen. Jag satt länge med Figma och ”duttade” med färger och typsnitt. Sedan planerade jag noga för mapp- och filstrukturen innan jag ens startade upp projektet för att börja koda. Den långa förberedelsen och den goda planeringen har gjort stor nytta. Jag behöver bara och göra fler projekt så är jag säker på att de många filerna och mapparna kommer att kännas hemtama så småningom.

AI-uttalande

Vid projektstart bad jag chat-GPT om ett förslag till en struktur tillsammans med lite korta förklaringar till de olika delarna i förslaget. Jag ser nu i efterhand att jag har vissa delar i mitt projekt som också fanns i förslaget, men inte andra. Jag tycker att det var hjälpsamt att se ett sådant förslag för att förbereda mig på den utmanande komplexiteten (se reflektion ovan) även om jag inte gjort precis så som föreslogs.

Jag har under projektets gång använt mig flitigt av det inbyggda AI-verktyget i VS code. Jag tycker att det ibland är hjälpsamt, men oftast blir jag irriterad eftersom jag tycker att den spottar ur sig oreflekterade förslag och råd som inte passar in i sammanhanget och för den nivå som projektet har vid frågetillfället.

Däremot har jag kunnat återanvända en del kod från våra code-alongs, men sammanfattningsvis kan jag uppriktigt säga att min kod är skriven av mig själv.

Länkar

https://www.figma.com/design/sLnbAcppL5WLZHsr6sowOg/BookClub?node-id=0-1&p=f&m=draw (troligen inte publikt tillgängligt)

https://github.com/Nomadity1/bokhyllan (publikt)